

Base R Graphics — devices & par() cheatsheet

Data Visualization with R · viz-base.rsquaredacademy.com · CC BY-NC-SA 4.0

DEVICES

Devices are a stack. Opening one makes it active; closing it returns to the one underneath. Every open device must be closed or output lands somewhere you did not choose — usually `Rplots.pdf`.

<code>png(f, width, height)</code> <code>pdf(f, width, height)</code> <code>svg(f, width, height)</code> <code>jpeg(f, ..., tiff(...))</code> <code>dev.off()</code> <code>dev.cur()</code> / <code>dev.list()</code> <code>dev.new()</code> <code>capabilities("cairo")</code>	raster, units defaults to "px"; res is ignored unless units is physical vector, for print, width/height are inches vector, for web. Needs a cairo build raster, lossy / lossless close the newest device; returns its name and number which device is active / which are open push a copy of the current device onto the stack TRUE if <code>svg()</code> will work
--	--

```
# dev.off() belongs INSIDE the loop, not after it
for (g in groups) {
  png(file.path(out, paste0(g, ".png")), width = 1800, res = 150)
  plot(x[[g]], y[[g]])
  dev.off()
}
```

PAR() — MARGINS AND TEXT

`mar` and `oma` are measured in **lines of text**, so they scale with `cex` and `pointsize`. A margin tuned on screen is wrong in a high-res export unless you raise `pointsize` too.

<code>par(mar = c(b, l, t, r))</code> <code>par(oma = c(b, l, t, r))</code> <code>par(mgp = c(3, 1, 0))</code> <code>par(las = 0, 3)</code> <code>par(bty = "n")</code> <code>par(xaxs = "i")</code> <code>par(yaxs = "i")</code> <code>par(tck = -0.02)</code> <code>par(cex = 1.2)</code> <code>par(family = "mono")</code> <code>par(font.main = 2)</code> <code>las = 1</code> is worth setting on any chart with numeric ticks — it stops y-axis labels being drawn slanted.	plot margins, in lines of text outer margins, only for multi-panel figures margin lines, tick labels, axis title tick labels: parallel, horizontal, perpendicular, vertical box: "o", "t", "7", "u", " ", " ", "f", "r" "r" (default, 4% padding) or "i" (exact data range) same, for the y axis tick length; negative draws outside, 0 removes scale all plot text base font family point size, main text
--	--

PAR() — PANELS AND STATE

<code>par(mfrow = c(2, 2))</code> <code>par(mfcol = c(2, 2))</code> <code>layout(matrix, widths, heights)</code> <code>layout(1)</code> <code>par(new = TRUE)</code> <code>par(no.readonly = TRUE)</code> <code>op <- par(x) / par(op)</code> <code>par()</code> is device state and persists until changed. Capture and restore rather than hand-writing old values back — and in a function, wrap the restore in <code>on.exit()</code> so an error cannot leave the device mangled.	panels filled by row panels filled by column ragged panels; widths/heights in 0–1 cancel <code>layout()</code> , back to a single panel draw onto the current plot without clearing it capture all current settings, for restoring set and capture in one call, then restore
--	--

```
init <- par(no.readonly = TRUE) # snapshot everything
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))
plot(x1)
plot(x2)
par(init) # always restore
```

EXPORT RECIPES

<code>units = "in", res = 300</code> <code>pdf(onefile = FALSE)</code> <code>pdf(width = 6, height = 4)</code> <code>pointsize = 12</code> <code>file.info(f)\$size</code>	the combination to use for anything printed one file per page — safer for long batch jobs inches, not pixels raise it on small or high-res exports, or labels vanish print it in a batch loop; 0 means <code>dev.off()</code> was missed
--	--

COLOURBLIND-SAFE PALETTES (R >= 4.1)

```
palette.pals() # every built-in palette
palette.colors(palette = "Okabe-Ito") # 9 colours, colourblind-safe
palette.colors(palette = "Okabe-Ito", alpha = 0.5)
```

Use the hex values as **R prints them** — hand-typing them from memory is how palettes get subtly wrong. The first Okabe-Ito colour is black, so drop it with `[-1]` unless you want black as your first series.

```
# greyscale check: Rec. 601 luma, not the mean of the channels
rgb <- t(col2rgb(pal)) / 255
luma <- rgb %*% c(0.299, 0.587, 0.114)
grey <- rgb(luma, luma, luma)
```

If the greyscale version has series you cannot tell apart, colour was doing work it should not do alone. Add a second channel: `pch`, `lty`, or direct labels.

AXIS() — DRAWING YOUR OWN

<code>xaxt = "n" / yaxt = "n"</code> <code>axis(1, at, labels)</code> <code>axis(1, at, labels = FALSE)</code> <code>axis(4)</code> <code>axis(..., las = 1)</code> <code>outer = TRUE</code> Axis positions are plain numbers, so dates work by converting and formatting.	suppress the automatic axis first , or you draw two 1 bottom · 2 left · 3 top · 4 right ticks with no labels add a second, right-hand axis — usually a different unit force horizontal labels, e.g. for dates draw on the figure margin, not the plot region
---	--

```
when <- as.Date("2026-01-01") + 0:6 * 30
plot(when, sales, type = "b", xaxt = "n")
axis(1, at = as.numeric(when), labels = format(when, "%b %d"), las = 1)
```

DRAWING PRIMITIVES

<code>plot(x, y, type=)</code> <code>points()</code> / <code>lines()</code> <code>segments(x0,y0,x1,y1)</code> <code>arrows(x0,y0,x1,y1)</code> <code>polygon(x, y, col=)</code> <code>rect()</code> / <code>text()</code> <code>legend(x, y, legend)</code> <code>text(x, y, labels)</code> <code>mtext(text, side)</code> <code>abline(a, b)</code>	"p" points · "l" line · "b" both · "o" overplot · "h" histogram · "n" axes only add to the current plot without clearing it straight segments: error bars, caps, reference lines line with a head; args run from → to closed filled shape; CI bands via <code>c(x, rev(x))</code> boxes and labels at coordinates x/y coords, or "topleft", "bottomright", ... labels at coordinates; pos, offset, padj margin text; side 1–4 as for <code>axis()</code> reference line; also <code>v =</code> , <code>h =</code>
--	---

STATE YOU MUST RESTORE

Every call below changes device state that outlives the call itself. If you change it, restore it — or the next plot in the session silently inherits it.

<code>call</code> <code>par(mfrow / mfcol)</code> <code>par(mar/oma/mgp/las/bty)</code> <code>par(new = TRUE)</code> an open device <code>setwd()</code> <code>options()</code>	leaves behind multi-panel layout; <code>layout(1)</code> cancels it page geometry, persists until reset a later <code>plot()</code> draws on top instead of clearing output goes to the wrong place; <code>dev.off()</code> it inside a script, the working directory stays changed process-wide until changed back
---	--

Full explanations: Chapter 12, **Production Export & Graphical Devices** · viz-base.rsquaredacademy.com